

Part 1

```
In [25]: import pandas as pd
```

```
In [33]: data = pd.read_csv(r"E:\\pradnya's programming\\Domains\\DataAnalytics\\DA_clgAss\\
```

```
In [34]: data.head()
```

```
Out[34]:
```

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single

```
In [35]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      500 non-null   int64
1   Age             500 non-null   int64
2   Income          500 non-null   int64
3   SpendingScore   500 non-null   int64
4   Gender          500 non-null   object
5   Profession       500 non-null   object
6   MaritalStatus   500 non-null   object
dtypes: int64(4), object(3)
memory usage: 27.5+ KB
```

```
In [36]: data['Profession'].value_counts()
data['Age'] = data['Age'].astype(float)
```

```
In [37]: from sklearn.preprocessing import LabelEncoder
label_Encoder = LabelEncoder()
data['Gender'] = label_Encoder.fit_transform(data['Gender'])
data['Gender'].unique()

data['Profession'] = label_Encoder.fit_transform(data['Profession'])
data['Profession'].unique()

data['MaritalStatus'] = label_Encoder.fit_transform(data['MaritalStatus'])
data['MaritalStatus'].unique()
```

```
Out[37]: array([1, 0])
```

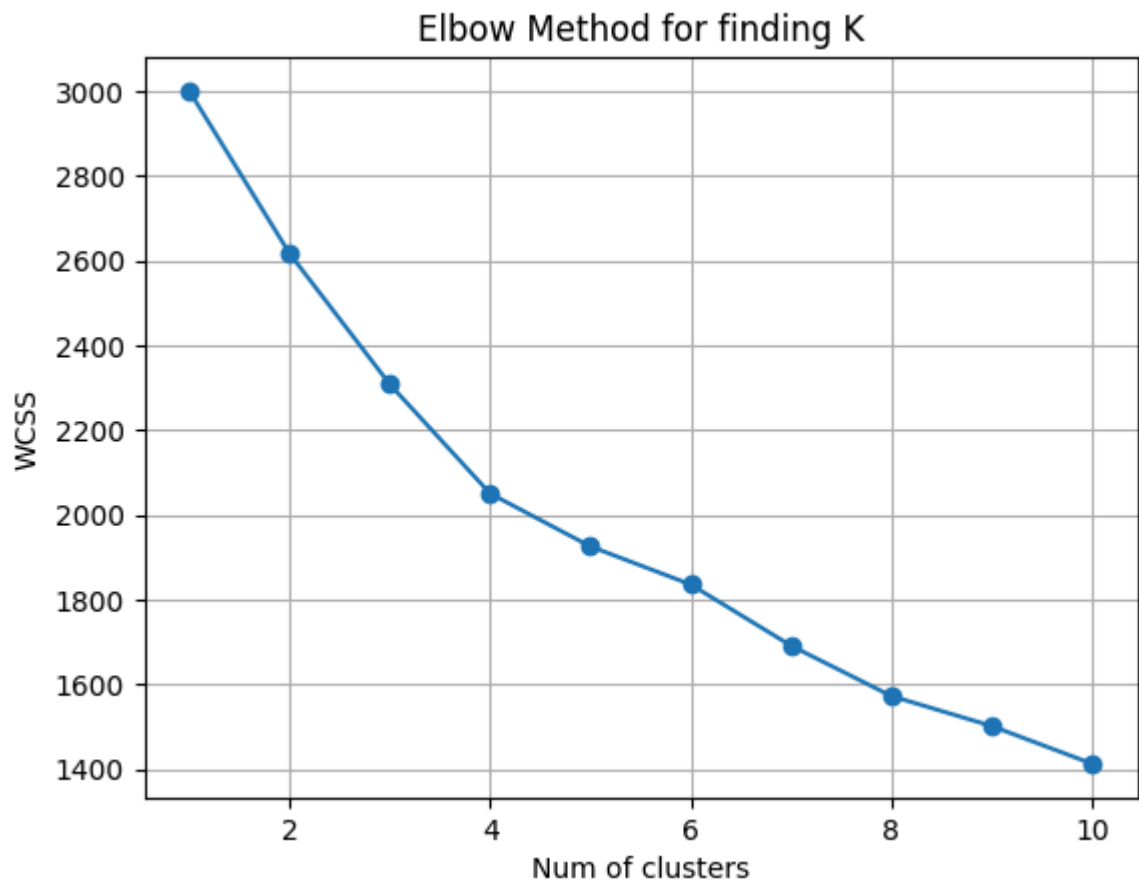
```
In [38]: features = ['Age', 'Income', 'SpendingScore', 'Gender', 'Profession', 'MaritalStatus']
x = data[features]
```

```
In [39]: from sklearn.preprocessing import StandardScaler
scalar = StandardScaler()
X_scaled = scalar.fit_transform(x)
```

```
In [40]: from sklearn.cluster import KMeans
wcss = []
for i in range(1,11):
    k = KMeans(n_clusters=i,random_state=42)
    k.fit(X_scaled)
    wcss.append(k.inertia_)
```

```
c:\Users\Pradnya\AppData\Local\Programs\Python\Python311\Lib\site-packages\joblib
\externals\loky\backend\context.py:136: UserWarning: Could not find the number of
physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by se
tting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
    File "c:\Users\Pradnya\AppData\Local\Programs\Python\Python311\Lib\site-package
s\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores
      cpu_info = subprocess.run(
                  ^^^^^^^^^^^^^^^^^^^
    File "c:\Users\Pradnya\AppData\Local\Programs\Python\Python311\Lib\subprocess.p
y", line 548, in run
      with Popen(*popenargs, **kwargs) as process:
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    File "c:\Users\Pradnya\AppData\Local\Programs\Python\Python311\Lib\subprocess.p
y", line 1024, in __init__
      self._execute_child(args, executable, preexec_fn, close_fds,
    File "c:\Users\Pradnya\AppData\Local\Programs\Python\Python311\Lib\subprocess.p
y", line 1493, in _execute_child
      hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
In [43]: import matplotlib.pyplot as plt
plt.plot(range(1,11),wcss,marker='o')
plt.title("Elbow Method for finding K")
plt.xlabel('Num of clusters')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```



```
In [44]: k = KMeans(n_clusters=6, random_state=42)
k.fit(X_scaled)
labels = k.labels_
centroids = k.cluster_centers_
```

```
In [50]: clusters = k.fit_predict(X_scaled)
data['clusters'] = clusters
```

```
In [51]: for i in range(6):
print(f"\n Cluster {i} sample data:")
print(data[data['clusters'] == i].head())
```

Cluster 0 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
0	1	56.0	31995	54	1	5	
18	19	39.0	10000	24	1	0	
19	20	70.0	34634	22	0	4	
21	22	41.0	31283	56	0	1	
23	24	47.0	28547	36	0	3	

	MaritalStatus	clusters
0	1	0
18	1	0
19	1	0
21	1	0
23	0	0

Cluster 1 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
1	2	69.0	44982	65	1	0	
2	3	46.0	42875	51	1	3	
6	7	38.0	31086	29	1	1	
8	9	36.0	81832	11	1	3	
10	11	28.0	27209	11	1	3	

	MaritalStatus	clusters
1	0	1
2	0	1
6	0	1
8	0	1
10	0	1

Cluster 2 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
4	5	60.0	76481	36	0	4	
5	6	25.0	56074	26	0	5	
7	8	56.0	63767	21	0	4	
22	23	61.0	74486	33	0	2	
28	29	29.0	67447	62	0	5	

	MaritalStatus	clusters
4	1	2
5	1	2
7	0	2
22	0	2
28	1	2

Cluster 3 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
3	4	32.0	40200	53	1	3	
12	13	41.0	69003	59	1	4	
15	16	57.0	61619	54	1	0	
24	25	55.0	43399	20	1	0	
32	33	66.0	52985	66	1	2	

	MaritalStatus	clusters
3	1	3
12	1	3
15	1	3
24	1	3
32	1	3

Cluster 4 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
9	10	40.0	65486	40	0	0	
11	12	28.0	42736	36	0	0	
17	18	20.0	49107	55	0	1	
20	21	19.0	46211	69	0	1	
25	26	19.0	51961	83	0	1	

	MaritalStatus	clusters
9	1	4
11	0	4
17	1	4
20	1	4
25	1	4

Cluster 5 sample data:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	\
14	15	53.0	56657	99	0	3	
16	17	41.0	36096	98	0	5	
26	27	38.0	71619	99	0	2	
30	31	61.0	35277	73	1	1	
38	39	64.0	73790	89	1	0	

	MaritalStatus	clusters
14	0	5
16	0	5
26	0	5
30	0	5
38	0	5

```
In [53]: import matplotlib.pyplot as plt
import numpy as np

# Example cluster names - adjust based on your analysis
# cluster_names = {
#     0: 'Young Spenders',
#     1: 'High Income - High Spending',
#     2: 'Savers',
#     3: 'Low Income - Low Spending'
# }

colors = ['blue', 'green', 'orange', 'purple']
k = len(np.unique(clusters))

# 2x2 grid setup
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for i in range(k):
    ax = axes[i]
    ax.scatter(X_scaled[clusters == i, 0], X_scaled[clusters == i, 1],
               color=colors[i], alpha=0.6)
    ax.scatter(centroids[i, 0], centroids[i, 1], marker='X', s=200, c='red', label=f'{cluster_names[i]}')
    # ax.set_title(f'{cluster_names[i]}')
    # ax.set_xlabel('Age (Standardized)')
    # ax.set_ylabel('Income (Standardized)')
    ax.legend()
    ax.grid(True)

# Hide extra axes if clusters < 4
```

```

for j in range(k, len(axes)):
    fig.delaxes(axes[j])

plt.suptitle('Customer Segmentation - ', fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

```

IndexError

Traceback (most recent call last)

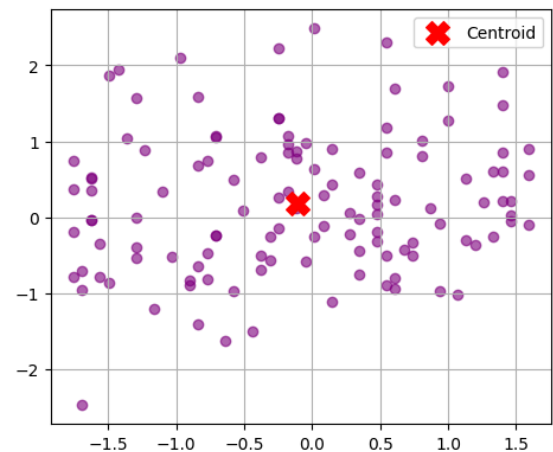
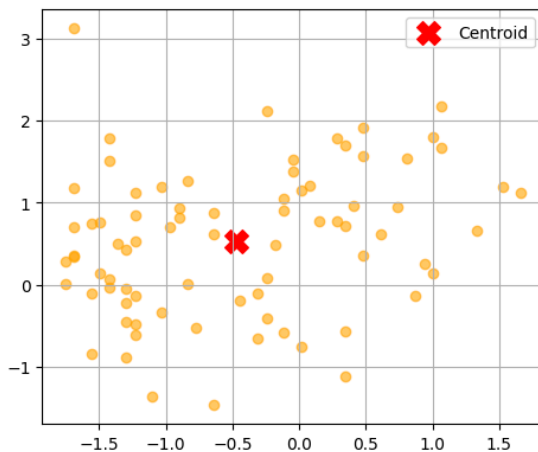
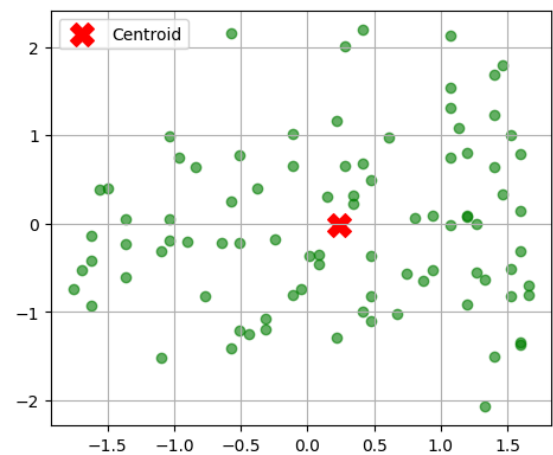
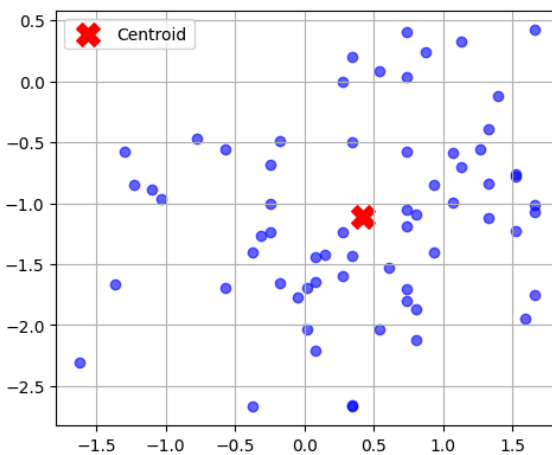
Cell In[53], line 20

```

17 axes = axes.flatten()
19 for i in range(k):
---> 20     ax = axes[i]
21     ax.scatter(X_scaled[clusters == i, 0], X_scaled[clusters == i, 1],
22               color=colors[i], alpha=0.6)
23     ax.scatter(centroids[i, 0], centroids[i, 1], marker='X', s=200, c='red', label='Centroid')

```

IndexError: index 4 is out of bounds for axis 0 with size 4



Part B

```

In [54]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

```

```

In [55]: data = pd.read_csv(r"E:\\pradnya's programming\\Domains\\DataAnalytics\\DA_clgAs
data

```

Out[55]:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single
...	...	...	...	...	...	...	...
495	496	43	65215	64	Female	Lawyer	Married
496	497	70	22386	82	Male	Artist	Married
497	498	40	30806	95	Male	Teacher	Single
498	499	26	40627	38	Female	Student	Married
499	500	29	50391	32	Male	Engineer	Married

500 rows × 7 columns

In [56]: `data.head()`

Out[56]:

	CustomerID	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	1	56	31995	54	Male	Teacher	Single
1	2	69	44982	65	Male	Artist	Married
2	3	46	42875	51	Male	Lawyer	Married
3	4	32	40200	53	Male	Lawyer	Single
4	5	60	76481	36	Female	Student	Single

In [57]: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   CustomerID      500 non-null    int64
1   Age             500 non-null    int64
2   Income          500 non-null    int64
3   SpendingScore   500 non-null    int64
4   Gender          500 non-null    object
5   Profession      500 non-null    object
6   MaritalStatus   500 non-null    object
dtypes: int64(4), object(3)
memory usage: 27.5+ KB
```

In [58]: `missing_values = data.isnull().sum()`
`missing_values`

```
Out[58]: CustomerID      0
Age      0
Income    0
SpendingScore  0
Gender    0
Profession  0
MaritalStatus  0
dtype: int64
```

```
In [59]: data_cleaned = data.drop(columns=["CustomerID"])
data_cleaned
```

```
Out[59]:
```

	Age	Income	SpendingScore	Gender	Profession	MaritalStatus
0	56	31995	54	Male	Teacher	Single
1	69	44982	65	Male	Artist	Married
2	46	42875	51	Male	Lawyer	Married
3	32	40200	53	Male	Lawyer	Single
4	60	76481	36	Female	Student	Single
...	...	...	...	...	...	...
495	43	65215	64	Female	Lawyer	Married
496	70	22386	82	Male	Artist	Married
497	40	30806	95	Male	Teacher	Single
498	26	40627	38	Female	Student	Married
499	29	50391	32	Male	Engineer	Married

500 rows × 6 columns

```
In [60]: categorical_cols = ['Gender', 'Profession', 'MaritalStatus']
data_encoded = pd.get_dummies(data_cleaned, columns=categorical_cols)
```

```
In [61]: data_encoded = data_encoded.dropna()
data_encoded
```


Out[61]:

	Age	Income	SpendingScore	Gender_Female	Gender_Male	Profession_Artist	Pro
0	56	31995	54	False	True	False	
1	69	44982	65	False	True	True	
2	46	42875	51	False	True	False	
3	32	40200	53	False	True	False	
4	60	76481	36	True	False	False	
...	...	...	...	...	...	...	...
495	43	65215	64	True	False	False	
496	70	22386	82	False	True	True	
497	40	30806	95	False	True	False	
498	26	40627	38	True	False	False	
499	29	50391	32	False	True	False	

500 rows × 13 columns



```
In [62]: kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(data_encoded[['Age', 'Income']])
data_encoded['Segment'] = kmeans.labels_
```

```
In [63]: data['Segment'] = data_encoded['Segment']
```

```
In [64]: plt.figure(figsize=(10, 6))
plt.scatter(data['Age'], data['Income'], c=data['Segment'], cmap='viridis', s=50)
plt.title('Customer Segmentation Based on Age and Income')
plt.xlabel('Age')
plt.ylabel('Income')
plt.colorbar(label='Segment')
plt.grid(True)
plt.show()
```

